QUANTUM BRANCH COMPILATION WITHOUT SEQUENTIALIZATION: CIRCUIT SIZE IS BOUNDED BY PROGRAM TIME

EMMANUEL HAINRY, ROMAIN PÉCHOUX, MÁRIO SILVA

Editorial note. Source https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2025.22.
© Emmanuel Hainry, Romain Péchoux and Mário Silva, CC BY4.0. Official publisher mathematical transcription with all algorithm and proof circuit graphics retained.

Allowed standard prerequisites (editorial).

- Quantum circuit semantics, controlled unitary composition and standard asymptotic complexity. Author §2 provides exact program/circuit semantics. Theorem11 FBQP completeness via an external function-algebra construction is not an input to the counted Theorem9 and is not counted separately.

1 Introduction

Motivation

Quantum computing is an emerging paradigm of computation, where quantum physical phenomena such as entanglement and superposition are used to obtain an advantage over classical computation. A testament to the richness of the field is the variety of computational models: quantum Turing machines [6], quantum circuits [31, 26], measurement-based quantum computation [7, 12], linear optical circuits [21], among others.

In recent decades, a lot of effort has been put into developing high-level quantum programming languages that allow programmers to abstract away from the technicalities of these low-level models [27, 3]. Toward that end, several verification techniques such as type systems [15] or categorical approaches for reasoning about program semantics [4, 19] have been studied and developed to ensure the physical reality of compiled programs, for example, by ensuring that they satisfy the properties of quantum mechanics such as the no-cloning theorem [1] and unitarity [14].

An important line of research in this area involves checking polytime termination of quantum programs [11, 29, 17], by showing that any program can be simulated by a polytime quantum Turing machine (QTM). As a consequence of Yao's Theorem [31], demonstrating that polynomial-time QTMs are computationally equivalent to uniform and poly-sized quantum circuit families, such programs can be instantiated by a uniform family of quantum circuits of polynomial size, i.e., with a polynomial number of qubits and gates. However, direct compilation techniques often fail to ensure this property, due to the use of quantum branching, where the flow in a conditional is determined by the state of a qubit.

As an illustrative example, consider a quantum case, of the shape

$$\text{qcase } \mathrm{q}_1, \ldots, \mathrm{q}_k \text{ of } \{i \to \mathrm{S}_i\}_{i \in \{0,1\}^k},$$

Date: doi:10.4230/LIPIcs.FSCD.2025.22; CC BY4.0.

which executes the superposition of the unitary transformations U_i implemented by statements S_i, depending on the state $|i\rangle$ of the control qubits $q_1, \ldots, q_k$. This statement can be simulated by a QTM that, depending on k symbols in its tape, performs the instructions of different QTMs M_i in superposition. Consequently, its runtime is the maximum runtime among all M_i ([6, Branching Lemma]). Instantiating this qcase statement on quantum circuits can be done sequentially (in-depth) or in parallel (in-width). An in-depth encoding consists in applying each controlled-U_i gate sequentially, thus implementing the unitary transformation $\sum_{i\in\{0,1\}^k} |i\rangle\langle i| \otimes U_i$. The depth of the resulting circuit is the sum of the depths of all U_i, which is exponential in the number k of control qubits. An alternative strategy, inspired by QRAM, allows for the execution of each U_i in parallel in the circuit, using controlled swaps to perform routing of qubit addresses in linear depth on the number of control qubits. This strategy has been recently applied in compilation techniques to automatically parallelize the branches of qcase statements and allow for optimal parallel time (depth) complexity [35].

Since the unitaries are parallelized, the circuit depth is the maximum among the depths of U_i, but this strategy results in circuits where the width (number of qubits) scales as the sum of the complexities of U_i. Consequently, this parallelization requires a number of ancillas that grows exponentially on the number of control qubits. In summary, any implementation, that is agnostic about the structure of each U_i will incur such an exponential cost; this is the case of qcase statements used in the preparation of arbitrary quantum states [34, 16].

Automatic implementations of quantum conditionals therefore produce circuits with a size that scales with the sum (rather than the maximum) of the complexity of each branch. This problem, coined branch sequentialization in [32], leads, in many cases, to an exponential blow-up in circuit complexity, forcing the programmer to rewrite and optimize their code in order to make use of the program structure and improve the complexity bound. A challenging issue is, therefore, to develop quantum programming languages that avoid branch sequentialization by ensuring the correct circuit complexity for quantum control statements while providing full use of quantum control to the programmer [32, 33].

Contribution

This paper solves the above issue by introducing a programming language, called pbp for Polynomially Branching Program, with quantum case and first-order recursive procedures, on which compilation of quantum conditional does not lead to an exponential blow-up in circuit size. Our main contributions are as follows:

■

We introduce a compilation strategy compile (Figures 8 and 9) into quantum circuits, show its soundness (Theorem 8), and show that it avoids branch sequentialization on pbp (Theorem 9), a language with restrictions on recursive calls to procedures. Toward that end, we formally define the time complexity $\mathrm{Time}_P : \mathbb{N} \to \mathbb{N}$ of a program P (Definition 1) as a map from the number n of qubits to the maximal number of procedures called during a program execution on the Hilbert space $\mathbb{C}^{2^n}$. The time complexity of a qcase statement is precisely the maximum of the time complexities of its branches and we show that branch sequentialization is avoided as the circuits generated by compile have size bounded asymptotically by the time complexity of the compiled program (Theorem 9).

■

A natural question concerns the expressive power of language pbp. We also solve this issue by showing that pbp is sound and complete for quantum polynomial time (Theorem 11). Consequently, any polytime quantum algorithm can be encoded by

a pbp program. We illustrate the methodology through well-known examples such as the program QFT realizing the quantum Fourier transform (Example 6).
■

We also discuss asymptotic bounds for a strict extension of pbp, where the reduction in the input sorted set passed to procedure calls can be arbitrary. We show that this extension only increases the circuit complexity of the no-branch-sequentialization case by a linear factor (Theorem 13).
■

We also show that compile strictly improves on existing compilation strategies on first-order quantum programs with quantum case [17] by a polynomial speedup of arbitrarily large degree (Table 1).

A bird's-eye view of our compilation strategy

We will now consider a simple illustrating example where branch sequentialization can exponentially worsen the size complexity of the compiled circuit, and show how our compilation strategy can preserve a polynomial bound.

The program PAIRS defined in Figure 1 consists in a simple call to procedure pairs (line 7) on a list $\bar{q}$ of unique qubits. Let $|\bar{q}|$ be the number of qubits in $\bar{q}$. By language design, the procedure pairs immediately terminates whenever $|\bar{q}| = 0$. The procedure enters a quantum case (line 2) when $|\bar{q}| \geq 2$, otherwise it applies a NOT gate to the single qubit (line 6). On line 2, the program will branch depending on the state $\bar{q}[1,2]$ of the first two qubits in $\bar{q}$. Out of all four cases (lines 2-5), pairs only performs an operation when the first two qubits are in state $|00\rangle$ or $|11\rangle$, in which case it performs a recursive call on $\bar{q} \ominus [1,2]$, the list obtained by removing the first and second qubits of $\bar{q}$.

```
1  decl pairs(q̄){
2        if |q̄| ≥ 2 then qcase q̄[1,2] of {00 → call pairs(q̄ ⊖ [1,2]);
3                                         01 → skip;
4                                         10 → skip;
5                                         11 → call pairs(q̄ ⊖ [1,2]); }
6        else q̄[1] *= NOT; }
7  :: call pairs(q̄);
```

Figure 1: Program PAIRS.

Given an input state $|xy\rangle$, with $x \in \{0,1\}^*$ and $y \in \{0,1\}$, pairs will apply a NOT gate to the last qubit y if and only if x is a string in the language $(00 \mid 11)^*$. Since pairs performs at most one call per branch, and consumes 2 qubits from its input while doing so, its time complexity is in $O(|\bar{q}|)$, when taking the qcase complexity as the maximum of each branch.

We now discuss the circuit compilation of pairs, when $|\bar{q}| \geq 2$. In Figure 2, we give three strategies (a), (b), and (c), exemplifying different approaches. Strategies (a) and (b) are automatic compilation strategies, which ignore the inner structure of the program – the fact that the bodies of the two non-skip branches of the qcase are identical and therefore encode the same unitary transformation. Figure 2(a) represents an in-depth strategy, whereas Figure 2(b) implements an in-width strategy, making use of different registers r_{00} and r_{11}, both initialized to $\left|0^{|\bar{q}|-2}\right\rangle$ to perform each branch in parallel and then recombine the results in the same register. These two implementations require two recursive calls to pairs and, consequently, their size complexity is the sum of the complexities of the two branches. These strategies are performing branch sequentialization and, in both cases, the compiled circuit has size exponential in $|\bar{q}|$. In contrast, Figure 2(c) avoids this exponential blow-up,

making use of the fact that the two branches are identical. This strategy is able to merge the recursive calls into a single procedure call with the use of one ancilla. This is precisely the type of strategy used by our compilation algorithm compile (Section 3). Although this example is relatively simple due to similar recursive calls, compile allows us to deal with more involved situations, where branches are not identical.

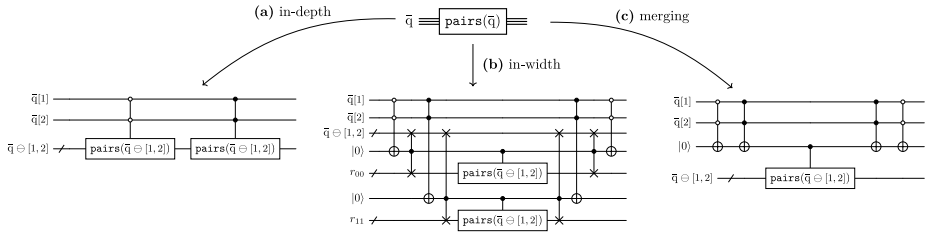


Figure 2: Compilation strategies.

Related work

Resource optimization in low-level models of quantum computation is a well-studied subject. Given a specific quantum circuit, it is possible to reduce its number of gates (or at least its number of non-Clifford gates) via techniques such as gate substitution and graph rewriting [25, 23, 20, 13]. The study of resource optimization in the asymptotic scenario is a relatively young research area as it involves reasoning about families of circuits and program structure. Different implicit characterizations of quantum complexity classes have been developed using a lambda-calculus [11], function algebras [29, 30] and a first-order programming language [17]. The last of these provided a compilation strategy into quantum circuits with bounds on the circuit size based on the syntactic restrictions placed on the programs.

Compilation strategies for the quantum control statement (also called quantum switch case or quantum multiplexer) have been studied, for instance, in state preparation [22], appearing in quantum machine learning [18] and Hamiltonian simulation [8]. These techniques either focus on optimizing number of qubits (circuit width) [34] or circuit depth [28], but in all cases correspond to circuits of exponential size. In order to improve on these bounds, one must restrict the set of programs to those that admit an efficient implementation, which can be deduced from the program structure. Optimized compilation techniques in that scenario can then be judged on expressivity and completeness: how easily can one write a program while ensuring the syntactical restrictions? Do these restrictions encompass all efficient programs? The field of implicit computational complexity is particularly well-suited to answer these questions [9, 10].

2 First-Order Programming Language with Quantum Case

In this section, we introduce a programming language with quantum control and first-order recursive procedures. After introducing its syntax and semantics, we introduce a restriction pbp (for Polynomially Branching Programs) on which branch sequentialization can be avoided.

(Integers)	$\mathrm{i}, \mathrm{j}, \mathrm{k}$	::=	$\mathrm{x} \mid n \mid \mathrm{i} \pm n \mid \lvert\mathrm{s}\rvert$
(Booleans)	b	::=	$\mathrm{i} \geq \mathrm{i} \mid \cdots \mid \mathrm{b} \wedge \mathrm{b} \mid \cdots$
(Qubits)	q	::=	$\mathrm{s}[\mathrm{i}]$
(Sorted sets)	s	::=	$\bar{\mathrm{q}} \mid \mathrm{s} \ominus [\mathrm{i}]$
(Statements)	S	::=	skip; $\mid$ q $*\!=$ $\mathrm{U}^f(\mathrm{j})$; $\mid$ S S $\mid$ if b then S else S $\mid$ qcase q of $\{0 \to \mathrm{S}, 1 \to \mathrm{S}\}$ $\mid$ call proc(s);
(Procedure declarations)	D	::=	$\varepsilon \mid$ decl proc($\bar{\mathrm{q}}$){S}, D
(Programs)	P($\bar{\mathrm{q}}$)	::=	D :: S

Figure 3: Syntax of programs.

2.1 Syntax of Programs

The language includes four basic datatypes for expressions, whose corresponding expressions are described in Figure 3.

■

Integers: Integer expressions are variables x, constant $n \in \mathbb{N}$, an addition by a constant $\mathrm{i} \pm n$, or the size $|\mathrm{s}|$ of a sorted set s.

■

Booleans: Such expressions b are defined in a standard way.

■

Qubit: qubits expressions are of the shape s[i] which denotes the i-th qubit in sorted set s.

■

Sorted sets: lists of unique (i.e., non-duplicable) qubit pointers. Sorted-set expressions s are either variables $\bar{\mathrm{q}}$ or $\mathrm{s} \ominus [\mathrm{i}]$, the sorted set s where the i-th element has been removed.

Let $\mathrm{s}[\mathrm{i}_1, \ldots, \mathrm{i}_n]$ be a shorthand for $\mathrm{s}[\mathrm{i}_1], \ldots, \mathrm{s}[\mathrm{i}_n]$. We also define syntactic sugar for pointing to the n-th last qubit in a sorted set, by defining for any $n \geq 1$, $\bar{\mathrm{q}}[-n] \triangleq \bar{\mathrm{q}}[|\bar{\mathrm{q}}| - n + 1]$.

A program $\mathrm{P} \triangleq \mathrm{D} :: \mathrm{S}$ is defined in Figure 3 as a list of (possibly recursive) procedure declarations D, followed by a program statement S.

Let Procedures be an enumerable set of procedure names. We write $\mathrm{proc} \in \mathrm{P}$ to denote that proc appears in P. Each procedure of name $\mathrm{proc} \in \mathrm{P}$ is defined by a (unique) procedure declaration $\mathrm{decl}\ \mathrm{proc}(\bar{\mathrm{q}})\{\mathrm{S}\} \in \mathrm{D}$, which takes a sorted set $\bar{\mathrm{q}}$ as input parameter and executes the procedure statement S. We sometimes write $\mathrm{S}^{\mathrm{proc}}$ to explicitly refer to the procedure statement S of proc.

Statements include the no-op instruction, unitary application, sequences, conditional, quantum case, and procedures calls. For the sake of universality [5], in a unitary application $\mathrm{q} \mathrel{*}= \mathrm{U}^f(\mathrm{j});$, $\mathrm{U}^f(\mathrm{j})$ is a unitary transformation that can take an integer j and a polynomial-time approximable [2] total function $f \in \mathbb{Z} \to [0, 2\pi)$ as optional arguments. The f and i can be omitted when they are not useful, as in a NOT gate.

The quantum conditional $\mathrm{qcase}\ \mathrm{s}[\mathrm{i}]\ \mathrm{of}\ \{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}$ allows branching by executing statements S_0 and S_1 in superposition according to the state of qubit s[i]. The procedure call $\mathrm{call}\ \mathrm{proc}(\mathrm{s});$ runs procedure proc with sorted set expression s. The quantum conditional can be extended to multiple qubits in a standard way as used in Figure 1: $\mathrm{qcase}\ \mathrm{q}_1, \mathrm{q}_2\ \mathrm{of}\ \{00 \to \mathrm{S}_{00}, 01 \to \mathrm{S}_{01}, 10 \to \mathrm{S}_{10}, 11 \to \mathrm{S}_{11}\}$ is a shorthand notation for $\mathrm{qcase}\ \mathrm{q}_1\ \mathrm{of}\ \{0 \to \mathrm{qcase}\ \mathrm{q}_2\ \mathrm{of}\ \{0 \to \mathrm{S}_{00}, 1 \to \mathrm{S}_{01}\}, 1 \to \mathrm{qcase}\ \mathrm{q}_2\ \mathrm{of}\ \{0 \to \mathrm{S}_{10}, 1 \to \mathrm{S}_{11}\}\}$.

We also make use of some syntactic sugar to describe statements encoding constant-time quantum operations. For instance, the CNOT, SWAP, as well as a controlled-phase shift gate are defined by:

$\mathrm{CNOT}(\mathrm{q}_1, \mathrm{q}_2) \triangleq \mathrm{qcase}\ \mathrm{q}_1\ \mathrm{of}\ \{0 \to \mathrm{skip};, 1 \to \mathrm{q}_2 \mathrel{*}= \mathrm{NOT};\}$ $\mathrm{SWAP}(\mathrm{q}_1, \mathrm{q}_2) \triangleq \mathrm{CNOT}(\mathrm{q}_1, \mathrm{q}_2)\ \mathrm{CNOT}(\mathrm{q}_2, \mathrm{q}_1)\ \mathrm{CNOT}(\mathrm{q}_1, \mathrm{q}_2)$ $\mathrm{CPHASE}(\mathrm{q}_1, \mathrm{q}_2, \mathrm{i}) \triangleq \mathrm{qcase}\ \mathrm{q}_1\ \mathrm{of}\ \{0 \to \mathrm{skip};, 1 \to \mathrm{q}_2 \mathrel{*}= \mathrm{Ph}^{\lambda x.\pi/2^{x-1}}(\mathrm{i});\}$

Given a program $\mathrm{P} \triangleq \mathrm{D} :: \mathrm{S}$, the call relation $\to_{\mathrm{P}} \subseteq \text{Procedures} \times \text{Procedures}$ is defined for any two procedures $\mathrm{proc}_1, \mathrm{proc}_2 \in \mathrm{P}$ as $\mathrm{proc}_1 \to_{\mathrm{P}} \mathrm{proc}_2$ whenever $\mathrm{proc}_2 \in \mathrm{S}^{\mathrm{proc}_1}$. The relation $\succeq_{\mathrm{P}}$ is then the transitive closure of $\to_{\mathrm{P}}$, and $\sim_{\mathrm{P}}$ denotes the equivalence relation where $\mathrm{proc}_1 \sim_{\mathrm{P}} \mathrm{proc}_2$ if $\mathrm{proc}_1 \succeq_{\mathrm{P}} \mathrm{proc}_2$ and $\mathrm{proc}_2 \succeq_{\mathrm{P}} \mathrm{proc}_1$ both hold.

A procedure proc is recursive whenever $\mathrm{proc} \sim_{\mathrm{P}} \mathrm{proc}$ holds. The strict order $\succ_{\mathrm{P}}$ is defined as $\mathrm{proc}_1 \succ_{\mathrm{P}} \mathrm{proc}_2$ if $\mathrm{proc}_1 \succeq_{\mathrm{P}} \mathrm{proc}_2$ and $\mathrm{proc}_1 \not\sim_{\mathrm{P}} \mathrm{proc}_2$ both hold.

2.2 Semantics of Programs

Let $\mathbb{B}$ denote the set of Booleans and $\mathcal{L}(\mathbb{N})$ denote the set of lists of natural numbers, [] being the empty list. We interpret each basic type as follows:

$[\![\text{Integers}]\!] \triangleq \mathbb{Z}$ $[\![\text{Booleans}]\!] \triangleq \mathbb{B}$ $[\![\text{Qubits}]\!] \triangleq \mathbb{N}$ $[\![\text{Sorted Sets}]\!] \triangleq \mathcal{L}(\mathbb{N})$ Qubits are interpreted as integers (pointers) and sorted sets are interpreted as lists of pointers. For each basic type τ, the semantics of expressions, given in Figure 5, is described in a standard way as a function

$$\Downarrow_{[\![\tau]\!]} \in \tau \times \mathcal{L}(\mathbb{N}) \to [\![\tau]\!].$$

$(\mathrm{e}, l) \Downarrow_{[\![\tau]\!]} v$ holds when expression e of type τ evaluates to the value $v \in [\![\tau]\!]$ under the context $l \in \mathcal{L}(\mathbb{N})$. l is simply the sorted set of qubit pointers considered when evaluating e. For instance, we have that $(\bar{\mathrm{q}}[2], [1,4,5]) \Downarrow_{\mathbb{N}} 4$ (the second qubit has index 4), $(\bar{\mathrm{q}} \ominus [4], [1,4,5]) \Downarrow_{\mathcal{L}(\mathbb{N})} []$ ([] is used for errors on type $\mathcal{L}(\mathbb{N})$), $(\bar{\mathrm{q}}[4], [1,4,5]) \Downarrow_{\mathbb{N}} 0$ (index 0 encodes error on type $\mathbb{N}$), and $(\bar{\mathrm{q}} \ominus [3], [1,4,5]) \Downarrow_{\mathcal{L}(\mathbb{N})} [1,4]$ (third qubit removed).

$$\frac{}{(\textbf{skip};, |\psi\rangle, A, l) \xrightarrow{0} (\top, |\psi\rangle, A, l)} \qquad \frac{(\mathrm{q}, l) \Downarrow_{\mathbb{N}} n \notin A}{(\mathrm{q} \mathrel{*}= \mathrm{U}^f(\mathrm{j});, |\psi\rangle, A, l) \xrightarrow{0} (\bot, |\psi\rangle, A, l)}$$

$$\frac{(\mathrm{q}, l) \Downarrow_{\mathbb{N}} n \in A \qquad (\mathrm{j}, l) \Downarrow_{\mathbb{Z}} k}{(\mathrm{q} \mathrel{*}= \mathrm{U}^f(\mathrm{j});, |\psi\rangle, A, l) \xrightarrow{0} (\top, I_{2^{n-1}} \otimes [\![\mathrm{U}]\!](f)(k) \otimes I_{2^{t(|\psi\rangle)-n}} |\psi\rangle, A, l)}$$

$$\frac{(\mathrm{S}_1, |\psi\rangle, A, l) \xrightarrow{m_1} (\top, |\psi'\rangle, A, l) \qquad (\mathrm{S}_2, |\psi'\rangle, A, l) \xrightarrow{m_2} (\diamond, |\psi''\rangle, A, l)}{(\mathrm{S}_1\ \mathrm{S}_2, |\psi\rangle, A, l) \xrightarrow{m_1+m_2} (\diamond, |\psi''\rangle, A, l)}$$

$$\frac{(\mathrm{S}_1, |\psi\rangle, A, l) \xrightarrow{m} (\bot, |\psi\rangle, A, l)}{(\mathrm{S}_1\ \mathrm{S}_2, |\psi\rangle, A, l) \xrightarrow{m} (\bot, |\psi\rangle, A, l)} \qquad \frac{(\mathrm{b}, l) \Downarrow_{\mathbb{B}} b \qquad (\mathrm{S}_b, |\psi\rangle, A, l) \xrightarrow{m_b} (\diamond, |\psi'\rangle, A, l) \qquad \diamond \in \{\top, \bot\}}{(\textbf{if}\ \mathrm{b}\ \textbf{then}\ \mathrm{S}_{\textbf{true}}\ \textbf{else}\ \mathrm{S}_{\textbf{false}}, |\psi\rangle, A, l) \xrightarrow{m_b} (\diamond, |\psi'\rangle, A, l)}$$

$$\frac{(\mathrm{q}, l) \Downarrow_{\mathbb{N}} n \in A \qquad \forall k \in \{0,1\},\ (\mathrm{S}_k, |\psi\rangle, A \backslash \{n\}, l) \xrightarrow{m_k} (\top, |\psi_k\rangle, A \backslash \{n\}, l)}{(\textbf{qcase}\ \mathrm{q}\ \textbf{of}\ \{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}, |\psi\rangle, A, l) \xrightarrow{\max_{k\in\{0,1\}} m_k} (\top, \sum_k |k\rangle_n\langle k|_n |\psi_k\rangle, A, l)}$$

$$\frac{(\mathrm{q}, l) \Downarrow_{\mathbb{N}} n \in A \qquad \forall k \in \{0,1\},\ (\mathrm{S}_k, |\psi\rangle, A \backslash \{n\}, l) \xrightarrow{m_k} (\diamond_k, |\psi_k\rangle, A \backslash \{n\}, l) \qquad \bot \in \{\diamond_0, \diamond_1\}}{(\textbf{qcase}\ \mathrm{q}\ \textbf{of}\ \{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}, |\psi\rangle, A, l) \xrightarrow{\max_{k\in\{0,1\}} m_k} (\bot, |\psi\rangle, A, l)}$$

$$\frac{(\mathrm{q}, l) \Downarrow_{\mathbb{N}} n \notin A}{(\textbf{qcase}\ \mathrm{q}\ \textbf{of}\ \{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}, |\psi\rangle, A, l) \xrightarrow{0} (\bot, |\psi\rangle, A, l)} \qquad \frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} []}{(\textbf{call}\ \mathtt{proc}(\mathrm{s});, |\psi\rangle, A, l) \xrightarrow{1} (\top, |\psi\rangle, A, l)}$$

$$\frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} l' \neq [] \qquad (\mathrm{S}^{\mathtt{proc}}, |\psi\rangle, A, l') \xrightarrow{m} (\diamond, |\psi'\rangle, A, l') \qquad \diamond \in \{\top, \bot\}}{(\textbf{call}\ \mathtt{proc}(\mathrm{s});, |\psi\rangle, A, l) \xrightarrow{m+1} (\diamond, |\psi'\rangle, A, l)}$$

Figure 4: Semantics of statements.

$$\frac{(\mathrm{e}, l) \Downarrow_{[\![\tau_1]\!]} m \qquad (\mathrm{d}, l) \Downarrow_{[\![\tau_2]\!]} n}{(\mathrm{e}\ \mathrm{op}\ \mathrm{d}, l) \Downarrow_{[\![\mathrm{op}]\!]([\![\tau_1]\!],[\![\tau_2]\!])} [\![\mathrm{op}]\!](m,n)}\ (\mathrm{Op}) \qquad \frac{(\mathrm{i}, l) \Downarrow_{\mathbb{Z}} n}{(\mathrm{U}^f(\mathrm{i}), l) \Downarrow_{\mathbb{C}^{2\times 2}} [\![\mathrm{U}^f]\!](n)}\ (\mathrm{Unit})$$

$$\frac{}{(n, l) \Downarrow_{\mathbb{Z}} n}\ (\mathrm{Cst}) \qquad \frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} [x_1, \ldots, x_m] \qquad (\mathrm{i}, l) \Downarrow_{\mathbb{Z}} k \in [1, m]}{(\mathrm{s} \ominus [\mathrm{i}], l) \Downarrow_{\mathcal{L}(\mathbb{N})} [x_1, \ldots, x_{k-1}, x_{k+1}, \ldots, x_m]}\ (\mathrm{Rm}_{\in})$$

$$\frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} [x_1, \ldots, x_n]}{(|\mathrm{s}|, l) \Downarrow_{\mathbb{Z}} n}\ (\mathrm{Size}) \qquad \frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} [x_1, \ldots, x_m] \qquad (\mathrm{i}, l) \Downarrow_{\mathbb{Z}} k \notin [1, m]}{(\mathrm{s} \ominus [\mathrm{i}], l) \Downarrow_{\mathcal{L}(\mathbb{N})} [\,]}\ (\mathrm{Rm}_{\notin})$$

$$\frac{}{(\bar{\mathrm{q}}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} l}\ (\mathrm{Var}) \qquad \frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} [x_1, \ldots, x_m] \qquad (\mathrm{i}, l) \Downarrow_{\mathbb{Z}} k \in [1, m]}{(\mathrm{s}[\mathrm{i}], l) \Downarrow_{\mathbb{N}} x_k}\ (\mathrm{Qu}_{\in})$$

$$\frac{(\mathrm{s}, l) \Downarrow_{\mathcal{L}(\mathbb{N})} [x_1, \ldots, x_m] \qquad (\mathrm{i}, l) \Downarrow_{\mathbb{Z}} k \notin [1, m]}{(\mathrm{s}[\mathrm{i}], l) \Downarrow_{\mathbb{N}} 0}\ (\mathrm{Qu}_{\notin})$$

Figure 5: Semantics of expressions.

Let $\mathcal{H}_{2^n}$ denote the Hilbert space of n qubits $\mathbb{C}^{2^n}$ and $\mathcal{P}(\mathbb{N})$ denote the powerset of $\mathbb{N}$.

A configuration c over n qubits is of the shape $(\mathrm{S}, |\psi\rangle, A, l)$, for some statement $\mathrm{S} \in \text{Statements} \cup \{\top, \bot\})$, $\top$ and $\bot$ being two special symbols denoting termination and error, respectively, $|\psi\rangle$ is a quantum state in $\mathcal{H}_{2^n}$, where $A \in \mathcal{P}(\mathbb{N})$ is the set of accessible qubit pointers, and where $l \in \mathcal{L}(\mathbb{N})$ is the list of qubit pointers under consideration. Let Conf_n, be the set of configurations over n qubits. The initial configuration in Conf_n of a program D :: S on input state $|\psi\rangle \in \mathcal{H}_{2^n}$ is $c_{init}(|\psi\rangle) \triangleq (\mathrm{S}, |\psi\rangle, \{1, \ldots, n\}, [1, \ldots, n])$. A final configuration can be defined in the same way as $c_{final}(|\psi\rangle) \triangleq (\top, |\psi\rangle, \{1, \ldots, n\}, [1, \ldots, n])$.

Each unitary transformation U of a unitary application $\bar{\mathrm{q}}[\mathrm{i}] \mathrel{*}= \mathrm{U}^f(\mathrm{j});$, comes with a function $[\![\mathrm{U}]\!]$ assigning a unitary matrix $[\![\mathrm{U}]\!](f)(n) \in \mathbb{C}^{2\times 2}$ to each integer n and polynomial-time approximable total function $f \in \mathbb{Z} \to [0, 2\pi)$. For example, the gates of the quantum Fourier transform can be defined by $R_n \triangleq [\![\mathrm{Ph}]\!](\lambda x.\pi/2^{x-1})(n)$ with $[\![\mathrm{Ph}]\!](f)(n) \triangleq \left(\begin{smallmatrix} 1 & 0 \\ 0 & e^{if(n)} \end{smallmatrix}\right)$. The other basic unary gates are the NOT and the R_Y gate.

The big-step semantics $\cdot \xrightarrow{\cdot} \cdot$ is defined in Figure 4 as a relation in $\bigcup_{n\in\mathbb{N}} \mathrm{Conf}_n \times \mathbb{N} \times \mathrm{Conf}_n$. Standard notations from quantum computation are used such as tensor product $\otimes$, $\langle\psi|$ for the conjugate transpose of $|\psi\rangle$, or given a dimension m, $|k\rangle_m \triangleq I_{2^{n-1}} \otimes |k\rangle \otimes I_{2^{m-n}}$ for $k \in \{0, 1\}$. In Figure 4, the set A of accessible qubits is used to ensure that unitary operations on qubits can be physically implemented. For example, to ensure reversibility, in a quantum branch qcase s[i] of $\{0 \to \mathrm{S}_0,\ 1 \to \mathrm{S}_1\}$, statements S_0 and S_1 cannot access s[i].

Definition 1.

The time complexity $\mathrm{Time}_{\mathrm{P}} : \mathbb{N} \to \mathbb{N}$ of a program $\mathrm{P} \triangleq \mathrm{D} :: \mathrm{S}$ is defined by

$$\mathrm{Time}_{\mathrm{P}}(n) \triangleq \max_{|\phi\rangle \in \mathcal{H}_{2^n}} \{m \in \mathbb{N} \mid \exists\, |\phi'\rangle \in \mathcal{H}_{2^n},\ c_{init}(|\phi\rangle) \xrightarrow{m} c_{final}(|\phi'\rangle)\}.$$

Intuitively, when $c \xrightarrow{m} c'$ holds, the time complexity m is an integer corresponding to the maximum number of procedure calls performed over each (classical and quantum) branch during the evaluation of a configuration $c \in \mathrm{Conf}_n$. We write $[\![\mathrm{P}]\!](|\psi\rangle) = |\psi'\rangle$, whenever $c_{init}(|\psi\rangle) \xrightarrow{m} c_{final}(|\psi'\rangle)$ holds for some m. If the program P terminates on any input (i.e., always reaches a final configuration) then $[\![\mathrm{P}]\!]$ is a total function on quantum states.

Example 2.

For the program PAIRS of Figure 1, $\mathrm{Time}_{\texttt{PAIRS}}(n) = \lfloor \frac{n}{2} \rfloor + 1$ since each procedure call removes two qubits until it reaches a sorted set $\bar{\mathrm{q}}$ such that $|\bar{\mathrm{q}}| \leq 1$ (depending on whether n is odd or even) and for both sizes there are no more procedure calls.

2.3 Polynomial Branching Programs

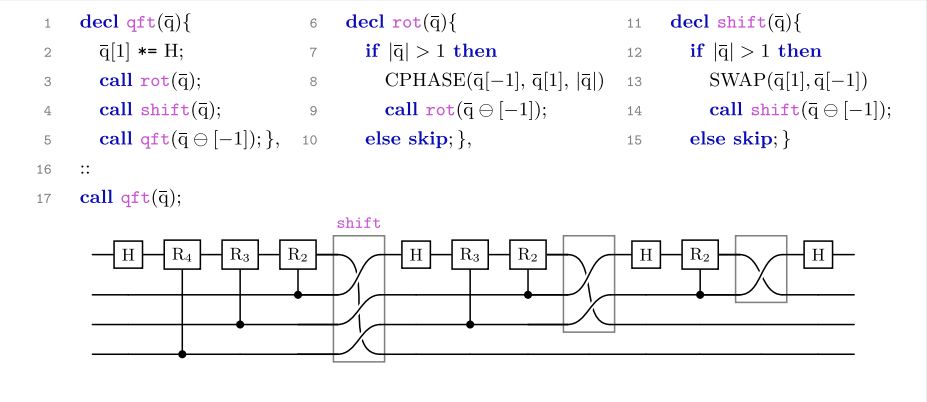


Figure 6: Program QFT for the quantum Fourier transform.

We define three restrictions on the programming language to consider only a strict subset, called pbp for Polynomial Branching Programs, on which branch sequentialization can be avoided. First, we define a well-foundedness criterion to consider only terminating programs. A program P is well-founded if each recursive procedure call removes at least one qubit in its parameter. wf denotes the set of well-founded programs. Then, we define a criterion to exclude programs with exponential runtime. Toward that end, the notion of width of a procedure proc in a program P is introduced.

Definition 3.

Given a program P, the width of a procedure proc $\in$ P, denoted $0pt_{\mathrm{P}}(\mathrm{proc})$, is defined as $0pt_{\mathrm{P}}(\mathrm{proc}) \triangleq w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}^{\mathrm{proc}})$, where $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S})$ is defined inductively as follows:

$w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{skip};) \triangleq 0$ $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{q} \mathrel{*}= \mathrm{U}^f(\mathrm{i});) \triangleq 0$, $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_1\ \mathrm{S}_2) \triangleq w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_1) + w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_2)$, $w_{\mathrm{P}}^{\mathrm{proc}}(\text{if b then } \mathrm{S}_0 \text{ else } \mathrm{S}_1) \triangleq \max(w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_0), w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_1))$, $w_{\mathrm{P}}^{\mathrm{proc}}(\text{qcase q of } \{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}) \triangleq \max(w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_0), w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_1))$, $w_{\mathrm{P}}^{\mathrm{proc}}(\text{call proc'}(\mathrm{s});) \triangleq \begin{cases} 1 & \text{if proc} \sim_{\mathrm{P}} \text{proc'}, \\ 0 & \text{otherwise.} \end{cases}$

Let $\mathrm{width}_{\leq 1}$ be the set of programs with procedures of width at most 1.

Programs of width 1 are inherently polynomial as they cannot perform an exponential number of procedure calls in sequence. However, the total number of calls in superposition (that is to say, across all branches of computation) may be exponential for such programs by the definition of the width for conditional and quantum case. For this reason, a compilation strategy whose circuits scale as the sum of the branches instead of their maximum can lead to circuits of exponential size.

Finally, for the purpose of our compilation process, we impose further syntactical conditions on programs. We restrict our attention to what we will call basic programs.

Definition 4.

A program is basic if there exists a sorted set s such that every procedure call is of the shape call proc($\bar{\mathrm{q}}$) or call proc(s). Let basic denote the set of basic programs.

basic programs are programs whose procedure calls are restricted to the full qubit list or to a fixed qubit list expression. This restriction simplifies the treatment of recursive procedure calls during compilation.

Definition 5 (Polynomially Branching Programs).
The set pbp of polynomially branching programs is defined as pbp $\triangleq$ wf $\cap$ width$_{\leq 1}$ $\cap$ basic.

Notice that pbp is strictly included in the programming language of [17], that roughly corresponds to wf $\cap$ width$_{\leq 1}$, where procedure calls can take an extra integer parameter. Since the language of [17] is polytime-sound, so is pbp (Theorem 11). The restriction to programs in pbp does not compromise the extensional completeness of the language as, by Theorem 11, pbp is complete for the class fbqp of functions in $\{0,1\}^* \to \{0,1\}^*$, i.e., functions that can be approximated with at least 2/3 probability by a quantum Turing machine running in polynomial time [6, 29].

Example 6.
The QFT program written in Figure 6 is in pbp. Indeed, the program QFT is in basic as calls are only performed on either $\bar{\mathrm{q}}$ or $\bar{\mathrm{q}} \ominus [-1]$. It is also in wf as all recursive calls are performed on parameter $\bar{\mathrm{q}} \ominus [-1]$. Finally, it is in width$_{\leq 1}$ as $0pt_{\texttt{QFT}}(\mathrm{qft}) = 0pt_{\texttt{QFT}}(\mathrm{rot}) = 0pt_{\texttt{QFT}}(\mathrm{shift}) = 1$.

3 Compilation Strategy

We now present the compilation algorithm from wf$\cap$width$_{\leq 1}$ to quantum circuits based on the merging strategy of Figure 2(c). Compilation is restricted to programs in wf$\cap$width$_{\leq 1}$ for two reasons. The well-foundedness criterion wf ensures that the compilation always terminates. The restriction to width$_{\leq 1}$ prevents exponential blow-up. Note however that, in Theorem 9, the extra basic restriction is required to avoid the branch sequentialization problem, that is, branch sequentialization is avoided on pbp.

3.1 Anchoring and Merging

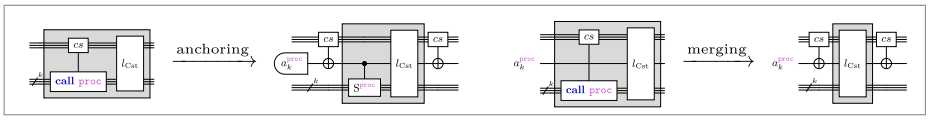


Figure 7: Anchoring and merging.

A control structure cs is a partial map in $\mathbb{N} \to \{0,1\}$. Intuitively, cs represents a map from qubit pointers to their control values for a controlled gate and will also be used as a shorthand notation in circuits. For $n \in \mathbb{N}$ and $k \in \{0,1\}$, let $cs[n := k]$ be the control structure obtained from cs by setting $cs(n) = k$. We denote by $dom(cs)$ the domain of the control structure and by $\cdot$, the control structure such that $dom(\cdot) = \emptyset$.

During the compilation of the program statement, every recursive call is handled as follows: if it is the first call with this procedure name and input size, an ancilla is created and its procedure statement is compiled under a control on this ancilla (anchoring), otherwise this procedure call has already been compiled and the control structure of the new call is sent into the corresponding ancilla (merging). Anchoring and merging are represented in Figure 7 as rewrite rules on quantum circuits for a procedure call call proc(s), cs denoting the control structure corresponding to the (possibly nested) quantum cases, where the procedure call occurs.

a_k^{proc} represents the ancilla introduced (in case of anchoring) or reused (in case of merging). The integer k refers to the number $|\mathrm{s}|$ of qubits in the procedure call. The grey box materializes the controlled statements left to compile.

3.2 The compile Algorithm

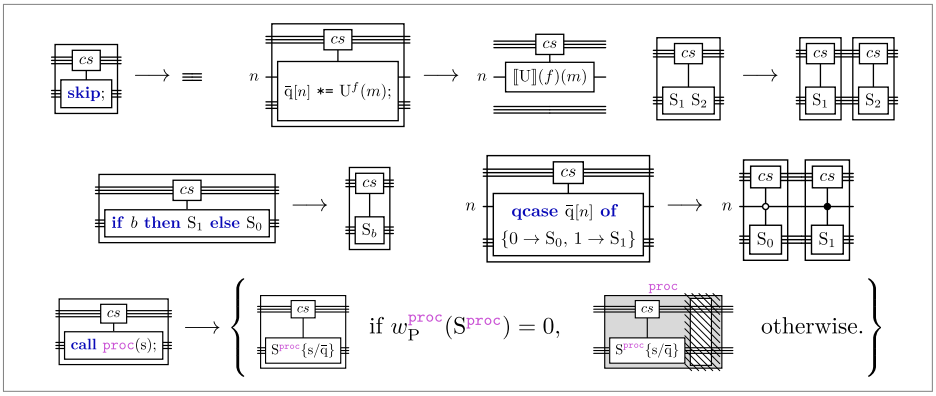
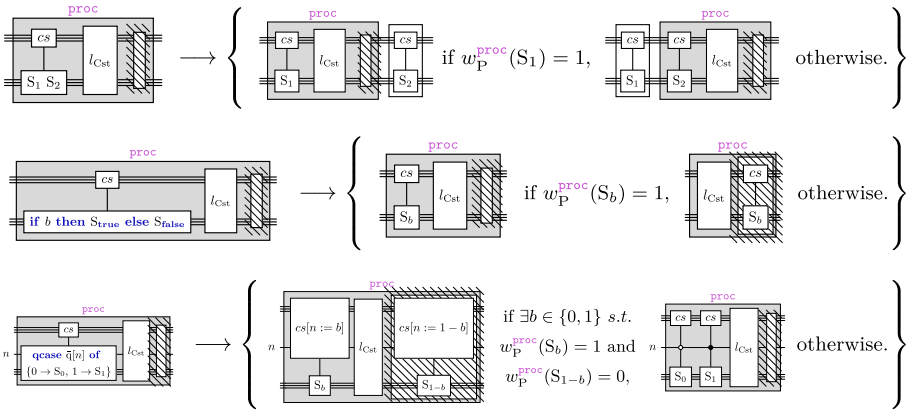


Figure 8: Rewrite rules of compile.

Let controlled statements be pairs of the shape (cs, S), for some control structure cs and some statement S. In the compilation process, controlled statements (cs, S) are used to represent a statement S that remains to be compiled into a quantum circuit C together with a control structure cs, representing the qubits controlling the compiled circuit C. The compilation algorithm compile is described by the rewrite rules of Figure 8. Generating the circuit corresponding to the program $\mathrm{P} = \mathrm{D} :: \mathrm{S}$ will consist in running compile on the controlled statement $(\cdot, \mathrm{S})$ for a fixed list of qubits pointers $[1, \ldots, n]$ (hence, an input of fixed size n). We denote by $\mathrm{compile}(\mathrm{P}, n)$ the circuit obtained for program P on size n. The algorithm standardly generates the quantum circuit corresponding to a $\mathrm{wf} \cap \mathrm{width}_{\leq 1}$ program inductively on the statement S. Indeed, in the rules of Figure 8 when compile is called on a given controlled statement (cs, S), an inductive call to compile is performed on controlled statements whose statements are sub-statements of S. The two base cases are the rules for the skip statement and the unitary application. In these cases, the compilation just outputs the identity circuit and a controlled gate computing the unitary, respectively. The rules for sequence and quantum case perform two inductive calls to compile on each branch of the statement. The rule for quantum case is the only rule that directly performs changes on the control structure. In the particular case of a call to a recursive procedure (i.e., when $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}^{\mathrm{proc}}) > 0$), compile calls the optimize subroutine to perform anchoring and merging. This call to optimize is highlighted in Figure 8 through the use of a shaded square ▣, which takes the procedure name proc as superscript. We call this process the optimization of procedure proc.



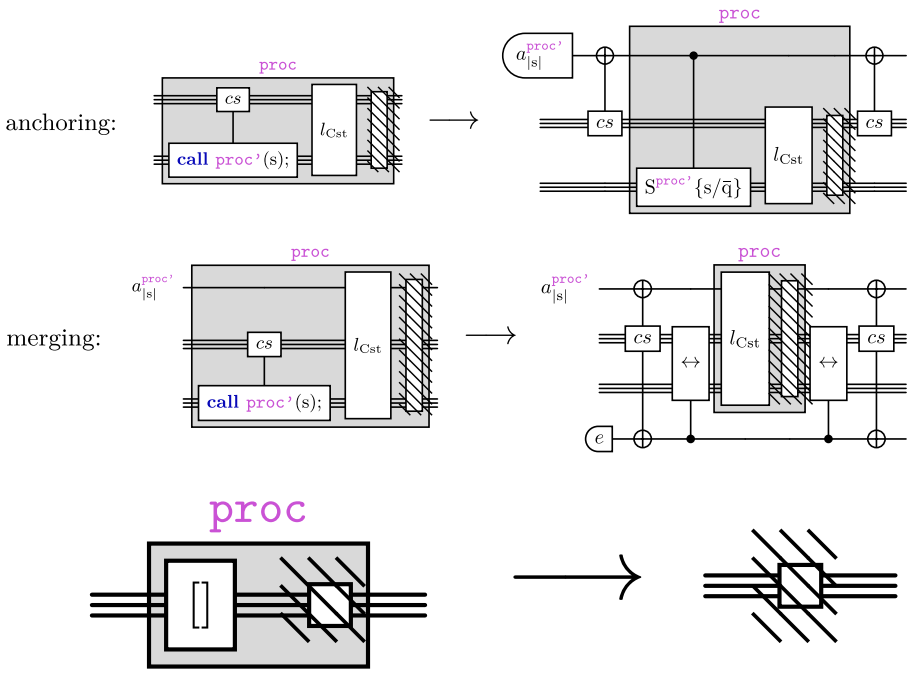


Figure 9: Rewrite rules of optimize.

The rewrite rules for the subroutine optimize on procedure proc are described in Figure 9. This subroutine takes two input parameters:

■ a first list l_{Cst} of controlled statements, the ones to be optimized,

■ a second list of controlled statements, called contextual list called l_{M} and denoted by ▨, consisting in controlled statements that are orthogonal to those in l_{Cst} and do not contain recursive procedure calls.

As described by the last rule of Figure 8, each initial call to optimize is performed on the singleton list containing one controlled statement $(cs, \mathrm{S}^{\mathrm{proc}}\{s/\bar{q}\})$ and a contextual circuit equal to the identity circuit.

In Figure 9, the rules are applied by considering the first controlled statement in the list l_{Cst}. We just specify the most interesting case below:

■ For a controlled statement $(cs, \mathrm{S}_1\ \mathrm{S}_2)$, two distinct rules can be applied. In the first scenario, where $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_1) = 1$, we have that S_1 contains a recursive procedure call and S_2 does not (this is a consequence of the width$_{\leq 1}$ condition in pbp), or the converse. Depending on this, the rule select on which control statement (cs, S_1) or (cs, S_2) to perform the optimization and, append the compiled circuit of the other controlled statement to the left or to the right.

■ For a controlled statement with an if statement, we precompute the boolean value b (which only depends on classical values such as the integer input and the register size), and according to whether the corresponding branch contains a recursive call or not, we either add the controlled statement to l_{Cst} or the contextual list.

■ For a controlled statement with a qcase statement, one or two of the branches are added to the list l_{Cst} depending on whether they are both recursive or not. The branch that is not recursive is added to the contextual list.

■

For a controlled statement with a procedure call call proc'(s); we perform either anchoring or merging depending on whether the ancilla $a_{|\mathrm{s}|}^{\mathrm{proc'}}$ exists or not. In the case of merging, we include a controlled permutation of qubit addresses (as described in Lemma 12) in order to match the input qubits of the different procedures. Notice that, for basic programs, the input qubits will match and therefore this permutation is trivial, and merging is as described in Figure 7.

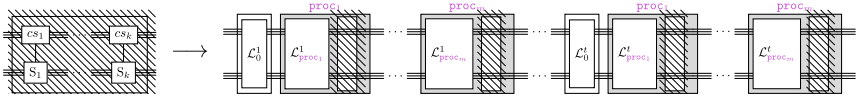
At the end of optimize, when $l_{\mathrm{Cst}} = []$, we rearrange the contextual list in the following way. Let $[(cs_1, \mathrm{S}_1, l_1), \ldots, (cs_k, \mathrm{S}_k, l_k)]$ be the state of the contextual list at the end of optimize. We may rewrite each controlled statement as a list of its atomic elements, with a transformation denoted seq. Let $(\mathrm{q}, l) \Downarrow_{\mathbb{N}} n$, then

$\mathrm{seq}(cs, \mathrm{skip}; , l) \triangleq []$, $\mathrm{seq}(cs, \mathrm{q} \mathrel{*}= \mathrm{U}^f(\mathrm{j}); , l) \triangleq [(cs, \mathrm{q} \mathrel{*}= \mathrm{U}^f(\mathrm{j}); , l)]$, $\mathrm{seq}(cs, \mathrm{S}_1\ \mathrm{S}_2, l) \triangleq \mathrm{seq}(cs, \mathrm{S}_1, l)@\mathrm{seq}(cs, \mathrm{S}_1, l)$, $\mathrm{seq}(cs, \mathrm{if\ b\ then\ S_{true}\ else\ S_{false}}, l) \triangleq \mathrm{seq}(cs, \mathrm{S}_b, l)$, if $(\mathrm{b}, l) \Downarrow_{\mathbb{B}} b$, $\mathrm{seq}(cs, \mathrm{qcase\ q\ of}\{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}, l) \triangleq \mathrm{seq}(cs[n := 0], \mathrm{S}_0, l)@\mathrm{seq}(cs[n := 1], \mathrm{S}_1, l)$, $\mathrm{seq}(cs, \mathrm{call\ proc(s)}; , l) \triangleq [(cs, \mathrm{call\ proc(s)}; , l)]$.

This separation of statements allows for a partitioning according to the type of procedure call appearing in the statement. Given a list of controlled statements $\mathcal{L}$, we define the list $[\mathcal{L}_0, \mathcal{L}_1, \ldots, \mathcal{L}_m]$ where, for procedures proc1, …, procm that are not mutually recursive.

$\mathcal{L}_0 \triangleq \{(cs, \mathrm{S}, l) \in \mathcal{L} : \nexists \mathrm{proc}$ such that $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}^{\mathrm{proc}}) = 1$ and $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}) = 1\}$. $\mathcal{L}_{\mathrm{proc}_i} \triangleq \{(cs, \mathrm{S}, l) \in \mathcal{L} : w_{\mathrm{P}}^{\mathrm{proc}_i}(\mathrm{S}) = 1\}, \quad i = 1, \ldots, m.$

Given our choice of procedures and the controlled statements obtained from seq, this partition is well-defined. Performing these two partitions (first in terms of sequential order of statements, and then according to procedure calls), we are able to rewrite:



The different instances of optimize contain calls to procedures that are mutually recursive, which will allow for further anchoring and merging.

3.3 Soundness of the algorithm

In this section, we discuss the validity of the compilation algorithm. One first observation should be that, given a pbp program, the compilation necessarily terminates. For instance, in compile (Figure 8), all rules besides the procedure call rewrite the controlled statement into either a circuit or into instances of compile of smaller statements. In the case of a procedure call, the rewriting of the procedure body produces a finite number of calls to procedures of lower recursive level.

In optimize, a recursive procedure will result in a finite number of calls to mutually recursive procedures – this is ensured by the well-foundedness condition wf, that requires that recursive procedure calls reduce the size of the input, therefore procedure calls either reduce the level of recursion or the number of available qubits.

The soundness of the compilation algorithm is ensured by an orthogonality invariant in optimize. Let cs, cs' be two control structures. We say that cs and cs' are orthogonal if there exists $i \in \mathrm{dom}(cs) \cap \mathrm{dom}(cs')$ such that $cs(i) = 1 - cs'(i)$. Two controlled statements are orthogonal if their control structures are orthogonal.

Lemma 7.

During the compilation of a program $\mathrm{P} \in \mathrm{wf} \cap \mathrm{width}_{\leq 1}$, for each optimization of a (recursive) procedure proc, all controlled statements in the union of list l and the contextual list l_{M} are pairwise orthogonal.

Proof.

This can be checked by inspection of each rewrite rule in Figure 9, as either the same control structure cs is kept in the list, or it is removed, or we consider two control structures, $cs[n = 0]$ and $cs[n = 1]$, that preserve the orthogonality relations of cs and are also mutually orthogonal due to the control over qubit n. ◀

This invariant ensures the validity of optimize, and the soundness of the compilation algorithm. It is also a consequence of the $\text{width}_{\leq 1}$ restriction in pbp, as by the definition of width, at most one recursive call may appear per branch of a recursive procedure. This ensures that two recursive calls on a given procedure always occur in orthogonal branches and can be simply combined in the same ancilla. Given a circuit C, we define its semantics $[\![C]\!]$ naturally as the composition of the semantics of each gate.

Theorem 8 (Soundness of compilation).
Given a program $\mathrm{P} \in \mathrm{wf} \cap \mathrm{width}_{\leq 1}$ and a quantum state $|\psi\rangle \in \mathcal{H}_{2^n}$ we have that $[\![\mathrm{compile}(\mathrm{P}, n)]\!](|\psi\rangle) = [\![\mathrm{P}]\!](|\psi\rangle)$.

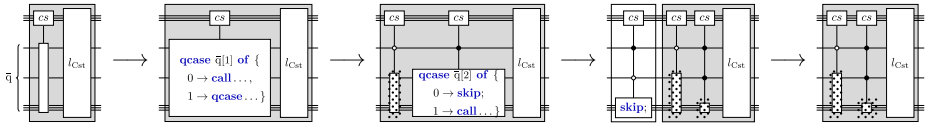
3.4 Illustrating Example

```
1  decl rec(q̄){
2    if |q̄| > 2 then qcase q̄[1] of{ 0 → call rec(q̄ ⊖ [1]);
3                                    1 → qcase q̄[2] of { 0 → skip;
4                                                        1 → call rec(q̄ ⊖ [1,2]); } }
5    else q̄[1] *= U; }
6  :: call rec(q̄);
```

Figure 10: Program REC.

We illustrate the compilation process with the program REC, of Figure 10. Notice that REC is in $\mathrm{wf} \cap \mathrm{width}_{\leq 1}$ but does not belong to pbp, as the procedure rec performs two recursive calls on different sorted sets $\bar{q} \ominus [1]$ and $\bar{q} \ominus [1, 2]$. This example thus illustrates that the compilation algorithm is not restricted to pbp.

Given that rec is a recursive procedure, its compilation is performed within optimize. We denote by the empty box □ the procedure statement $\mathrm{S}^{\mathrm{rec}}$ and by the dotted box ▩ the statement call rec(s);. Applying rewrite rules to rec we obtain the transitions:



The first step is obtained by applying the rule of if statements, where we assume that $|\bar{q}| > 2$. Afterwards, we apply twice the rule for the qcase statement, adding the two qubits $\bar{q}[1]$ and $\bar{q}[2]$ to the control structure. Finally, the compilation of the skip statement corresponds to the empty circuit. We denote by $\xrightarrow{\texttt{rec}}$ the application of this sequence of rewrite rules, which is used in Figure 11.

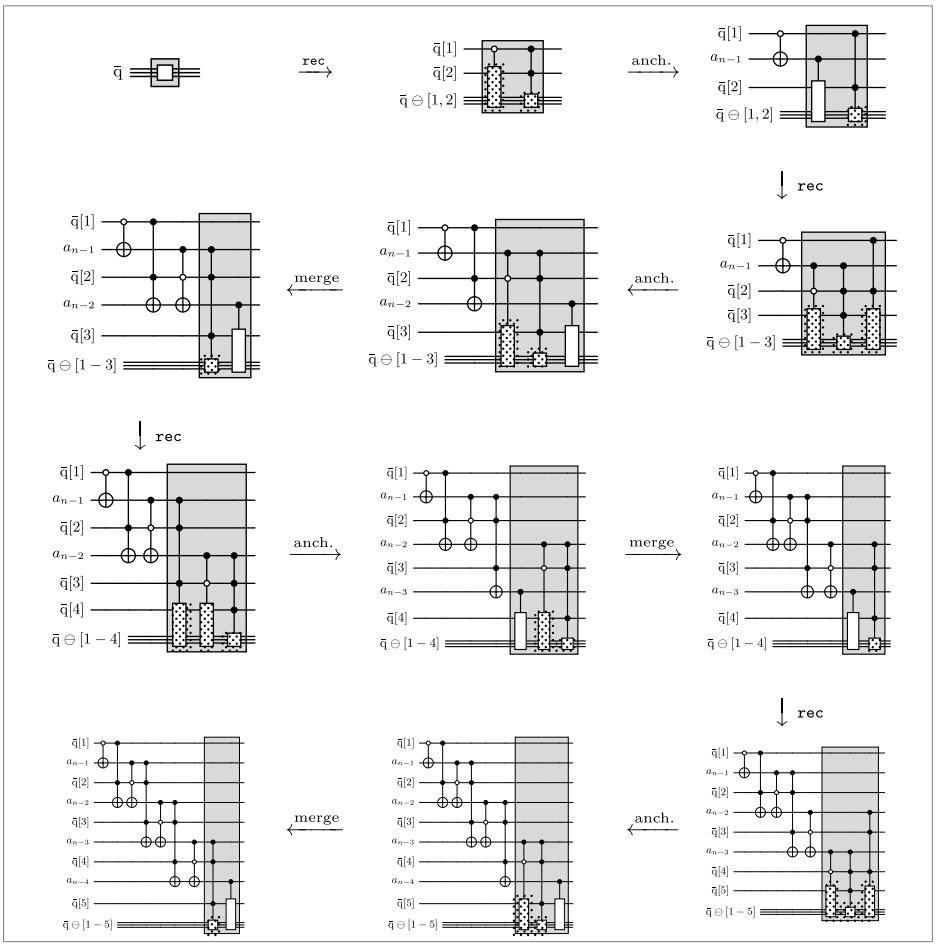


Figure 11: Example of anchoring and merging for the program REC in Figure 10.

One important thing to notice in Figure 11 is that the compilation strategy does not avoid branch sequentialization locally but rather asymptotically in the construction of the circuit. In other words, the qcase statement in rule rec does generate two instances of $\mathrm{S}^{\mathrm{rec}}$ in the circuit in sequence, one for each branch. However, the merging of calls to rec on inputs of the same size ensures that only one instance per input size needs to be compiled, and therefore this strategy achieves linear complexity in the number of gates.

4 Main Results

4.1 No Branch Sequentialization

First, we show that the problem of branch sequentialization is solved in pbp. For that purpose, we show that the size of quantum circuit obtained through compiling a pbp program is bounded asymptotically by the time complexity of the program. As the time complexity is the maximum of the complexity of the two branches of a quantum case statement, the branch sequentialization problem is avoided on pbp programs, as the size of the circuit is asymptotically equal to its time semantics. Given a circuit C, let $\#C$ denote its size, i.e., number of gates and wires.

One may notice that, in the rules of compile (Figure 8), the compilation of a qcase statement generates two controlled statements in sequence. Similarly, the rule of optimize (Figure 9) for qcase statements appends two controlled statements to the list l_{Cst} in the case where they are both recursive. This does not pose a

problem as, anchoring and merging will ensure that the asymptotic complexity of this statement will be given by the maximum complexity of the branches.

Example 10.
By Theorem 9 and Example 2, we have that $\#\text{compile}(\texttt{PAIRS}, n) = O(n)$.

4.2 Polynomial-Time Soundness and Completeness

Having shown in Theorem 9 that pbp programs can avoid branch sequentialization, we now turn to the question of whether they constitute an interesting fragment of quantum programs, given that it is restricted by the wf, $\text{width}_{\leq 1}$ and basic conditions. We show that the set of pbp programs is sound and complete for quantum polynomial time via an implicit characterization of fbqp, i.e. the set of classical functions that can be approximated with bounded error by a polytime quantum Turing machine [30, 17].

Since the considered programming language does not allow for qubit creation, in order to define functions where the output is larger than the input, we make use of polytime padding. A polytime padding function is a function $f : \{0,1\}^* \to \{0,1\}^*$ computable by a (classical) polytime Turing machine such that $f(x) = xy$, for some y depending only on the length of x (and not the value of x). Given a set $\mathcal{F}$ of programs, $[\![\mathcal{F}]\!]$ denotes the set of functions given by $[\![\mathrm{P}]\!] \circ f$, where $\mathrm{P} \in \mathcal{F}$, f is a polytime padding function, and $\circ$ is the standard function composition. For any $p \in (\frac{1}{2}, 1]$, we denote by $[\![\mathcal{F}]\!]_{\geq p}$ the set of functions in $[\![\mathcal{F}]\!]$ that approximate a classical function with probability at least p.

Theorem 11 (Soundness and Completeness).
$[\![\text{pbp}]\!]_{\geq \frac{2}{3}} = \text{fbqp}$.

4.3 Extension to Programs in $\text{wf} \cap \text{width}_{\leq 1}$

The basic restriction of pbp can be thought of as ensuring that qubits are consumed in a uniform way. As a consequence, any two procedure calls on inputs of size n will correspond to precisely the same n qubits. This guarantees that two compatible procedure calls can be merged simply by combining control structures in the same ancilla.

Without this constraint (i.e., on $\text{wf} \cap \text{width}_{\leq 1}$), we can use precisely the same compilation strategy but where merging is done by control-swapping registers into a single procedure call. For k instances of a procedure on input size n, this requires $k-1$ controlled swaps, each requiring $O(n)$ operations in the worst case. An example of merging two unitary gates U with controlled-swaps is given in Figure 12. If we allow for parallelization of gates in the circuit, this can be done in logarithmic depth, as shown in Lemma 12. An example of a controlled permutation is shown in Figure 13, where vertical dashed lines separate time slices where gates can be applied simultaneously.

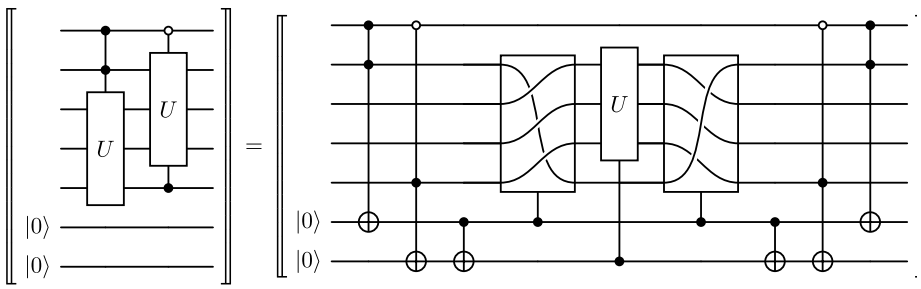


Figure 12: Merging orthogonal controlled statements.

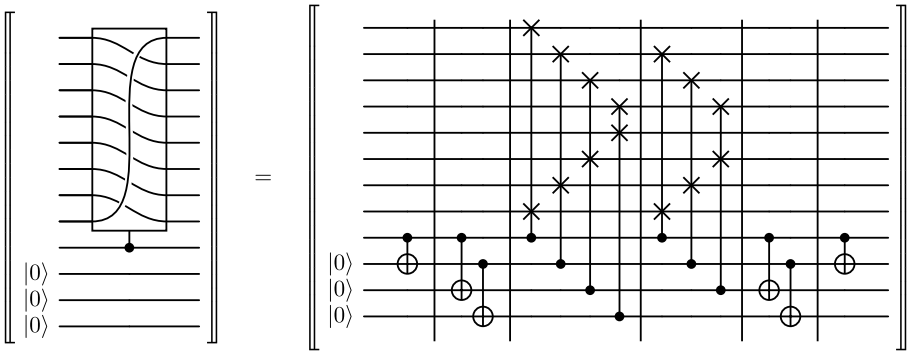


Figure 13: Implementation of a controlled permutation in log-depth.

Lemma 12.

Any controlled permutation on n qubits can be performed with a quantum circuit of size $O(n)$ and depth $O(\log n)$.

An extension of compile with controlled-swapping as in Figure 12 used for merging procedures ensures the following bound for $\mathrm{wf} \cap \mathrm{width}_{\leq 1}$ programs.

Theorem 13.

For $\mathrm{P} \in \mathrm{wf} \cap \mathrm{width}_{\leq 1}$, $\mathrm{compile}(\mathrm{P}, n)$ results in a quantum circuit of size $O(n \cdot \mathrm{Time}_{\mathrm{P}}(n))$ and depth $O(\log(n) \cdot \mathrm{Time}_{\mathrm{P}}(n))$.

4.4 Comparison with Existing Work

The compile algorithm strictly improves upon the size bounds of other compilation algorithms [17], as illustrated by Table 1. These bounds can be empirically tested using the implementation of the compiler available at gitlab.inria.fr/mmachado/pfoq-compiler. In some cases, such as Examples 16 and 15 given in Appendix B, we find families of programs where the gap in complexity (i.e. the degree of the polynomial) can be made arbitrarily large.

In k-Chained Substrings (Example 16), we consider the problem of detecting whether an input string contains substrings $y_1, \ldots, y_k$ appearing in that order. For a certain choice of substrings y_i, we define a pbp program of linear time complexity for which the compilation strategy in [17] results in circuits of size $\Theta(n^{2k})$. The problem $\mathrm{Sum}(r)$, given in Example 17, consists of checking if an input $x = x_1 \ldots x_n$ satisfies $\sum_{i=1}^{n} x_i = r$.

Table 1: Circuit size complexity bounds.

Circuit complexity Problem Example [17] compile Full Adder Example 15 $\Theta(n)$ $\Theta(n)$ Quantum Fourier Transform Example 6 $\Theta(n^2)$ $\Theta(n^2)$ k-Chained Substrings Example 16 $\Theta(n^{2k})$ $\Theta(n)$ $\mathrm{Sum}(r)$, $r \geq 1$ Example 17 $\Theta(n^r)$ $\Theta(n)$

Theorem 9 (No branch sequentialization).

Given a program $\mathrm{P} \in \mathrm{pbp}$ and input $n \in \mathbb{N}$, we have that $\#\mathrm{compile}(\mathrm{P}, n) = O(\mathrm{Time}_{\mathrm{P}}(n))$ holds.

Appendix A Omitted proofs

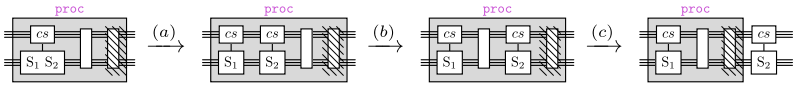
Theorem 8 (Soundness of compilation). [Restated, see original statement.]

Given a program $\mathrm{P} \in \mathrm{wf} \cap \mathrm{width}_{\leq 1}$ and a quantum state $|\psi\rangle \in \mathcal{H}_{2^n}$ we have that $[\![\mathrm{compile}(\mathrm{P}, n)]\!](|\psi\rangle) = [\![\mathrm{P}]\!](|\psi\rangle)$.

Proof.

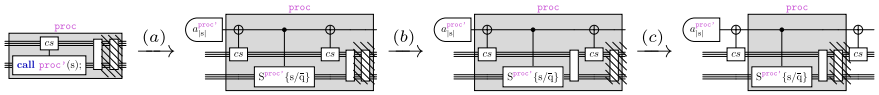
By induction on the structure of the program $\mathrm{P} = \mathrm{D} :: \mathrm{S}$ in $\mathrm{wf} \cap \mathrm{width}_{\leq 1}$. One can check by inspection of each case that the compile rules for non-recursive statements corresponds to the straightforward circuit semantics of quantum programs.

Likewise, the rewriting rules of optimize, given in Figure 9, can be easily checked using the orthogonality invariant of Lemma 7. For instance, consider the case of the sequential statement $\mathrm{S} = \mathrm{S}_1\ \mathrm{S}_2$. In the case where $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}_1) = 1$, we can derive the rule for the statement with the following steps. We omit the label l_{Cst} to simplify the reading.

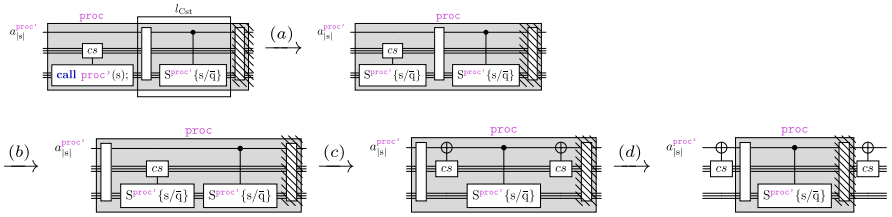


where (a) corresponds to the definition of the sequential statement, in (b) we make use of the fact that (cs, S_2) is orthogonal to all controlled statements in l_{Cst} and therefore can be commuted in the circuit, and in step (c) we likewise make use of orthogonality, in this case with the controlled statements in the contextual list. Other case can be inspected to follow a short sequence of steps, such as described for the sequential statement.

For instance, in the case of anchoring, we consider the following composition of rules

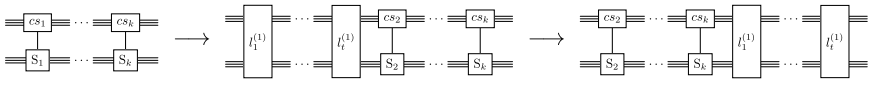


where, likewise, (a) corresponds to the typical circuit semantics of a procedure call (with an added anchoring ancilla), and (b) and (c) make use of the orthogonality of cs with the right side of the circuit. The validity of merging and also be checked:



Step (a) can be seen as the definition of the procedure call. Since we are in the merging scenario, a similar procedure call has already been performed and is anchored to its corresponding ancilla. Without loss of generality (since all controlled statements in l_{Cst} are orthogonal and therefore commute) we assume that this call appears at the end of l_{Cst}. Rule (b) simply indicates that since cs is orthogonal to other control structures in l_{Cst}, we may move the controlled statements so that they are adjacent, and where we apply step (c) to perform merging. Since cs is orthogonal to $\cdot[a_{|\mathrm{s}|}^{\mathrm{proc'}} = 1]$ the control is added to the ancilla as expected. Finally, orthogonality of cs with other control structures means that we may move the two controlled-NOTs to the edges of the circuit.

We now consider the validity of the rule for the contextual circuit. Consider a list of mutually-orthogonal controlled statements, where the sequential form of (cs_i, S_i) is given by the lists of controlled statements $l_1^{(i)}, \ldots, l_t^{(i)}$, then we have that:



The final step comes from the fact that all $l_j^{(1)}$ are orthogonal to cs_2, ..., cs_k since they include cs_1. This can be done for all other (cs_i, S_i), and in the end we obtain the arrangement $l_1^{(1)} \ldots l_1^{(k)} l_2^{(1)} \ldots l_2^{(k)} \ldots l_t^{(1)} \ldots l_t^{(k)}$.

Given a certain $1 \leq j \leq t$, we have that all controlled statements in $\cup_{i=1}^{k} l_j^{(i)}$ (that is, all controlled statements occurring in time j) are pairwise orthogonal. Therefore,

we may rearrange their order according to their recursivity, and in doing so we may consider each time separately. For instance, in time 1, let $\mathcal{L} \triangleq \cup_{i=1}^{k} l_1^{(i)}$, and let proc_1, $\mathrm{proc}_2, \ldots \mathrm{proc}_m$ denote procedures belonging to different recursion families. Then, we perform the partition:

$\mathcal{L}_0 \triangleq \{(cs, \mathrm{S}) \in \mathcal{L} :/\exists \mathrm{proc}$ such that $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}^{\mathrm{proc}}) = 1$ and $w_{\mathrm{P}}^{\mathrm{proc}}(\mathrm{S}) = 1\}$. $\mathcal{L}_i \triangleq \{(cs, \mathrm{S}) \in \mathcal{L} : w_{\mathrm{P}}^{\mathrm{proc}_i}(\mathrm{S}) = 1\}, \quad i = 1, \ldots, m$. By the definition of the sequential form of each controlled statement, we note that the partition is well-defined (e.g. there are no statements containing calls to more than one procedure). Therefore, we are able to rearrange $\mathcal{L}$ and perform calls to optimize on each $\mathcal{L}_{i\geq 1}$ and a call to compile on $\mathcal{L}_0$. After composing the obtained circuits, we get the rule given in Section 3. This concludes the proof. ◀

We introduce the notion of rank of a procedure for use in the proof of Theorem 9.

Definition 14 (Rank).

Set $\max(\emptyset) \triangleq 0$. Given a program P, the rank of a procedure $\mathrm{proc} \in \mathrm{P}$, denoted $rk_{\mathrm{P}}(\mathrm{proc})$, is defined as follows:

$$rk_{\mathrm{P}}(\mathrm{proc}) \triangleq \begin{cases} 0, & \text{if } \neg(\exists \mathrm{proc'},\ \mathrm{proc} \succeq_{\mathrm{P}} \mathrm{proc'}), \\ \max_{\mathrm{proc} \succeq_{\mathrm{P}} \mathrm{proc'}} \{rk_{\mathrm{P}}(\mathrm{proc'})\}, & \text{if } \exists \mathrm{proc'},\ \mathrm{proc} \succeq_{\mathrm{P}} \mathrm{proc'} \wedge \neg(\mathrm{proc} \sim_{\mathrm{P}} \mathrm{proc}), \\ 1 + \max_{\mathrm{proc} \succ_{\mathrm{P}} \mathrm{proc'}} \{rk_{\mathrm{P}}(\mathrm{proc'})\}, & \text{if } \mathrm{proc} \sim_{\mathrm{P}} \mathrm{proc}. \end{cases}$$

Theorem 9 (No branch sequentialization). [Restated, see original statement.]

Given a program $\mathrm{P} \in$ pbp and input $n \in \mathbb{N}$, we have that $\#\mathrm{compile}(\mathrm{P}, n) = O(\mathrm{Time}_{\mathrm{P}}(n))$ holds.

Proof.

The theorem can be shown by structural induction on the program body. All cases are straightforward except the one of the quantum control case. We make use of the following two facts regarding pbp programs:

(a)

The size of the circuit is directly proportional to its total number of unique procedure calls (in the sense required for merging), and

(b)

a recursive procedure call results in $O(n)$ unique calls to procedures of the same rank. This is because unique calls may only differ on procedure name (of which there is a constant amount) or input size (for which there is a linear number of possibilites).

Consider a quantum control statement $\mathrm{S} = \mathrm{qcase}\ \mathrm{q}\ \mathrm{of}\ \{0 \to \mathrm{S}_0, 1 \to \mathrm{S}_1\}$ appearing in optimize in the context of a recursive procedure proc. By (a), the circuit size for S_0 and S_1 are proportional to the (total) number of procedure calls in each statement, separately. We check that the number of ancillas created for S is bounded by the maximum number of ancillas between S_0 and S_1.

We proceed by induction on the rank r of the procedure. The base case of $r = 1$ is given by (b), and so we may consider $r > 1$. For the inductive case, we consider two scenarios:

■

$w_{\mathrm{proc}}^{\mathrm{P}}(\mathrm{S}_0) = w_{\mathrm{proc}}^{\mathrm{P}}(\mathrm{S}_1) = 1$. In this case, both S_0 and S_1 are of rank r, and all their recursive procedure calls may be merged. Therefore, the asymptotic number of such calls is bounded between the maximum between S_0 and S_1 (consider that, if there is no overlap between the ancillas needed, their number is still bounded

linearly). Applying the IH on the procedures of rank $r-1$ we obtain the desired result.

■

$w^{\mathrm{P}}_{\mathrm{proc}}(\mathrm{S}_0) = 0$ and $w^{\mathrm{P}}_{\mathrm{proc}}(\mathrm{S}_1) = 1$. In this case, S_0 contains calls to procedures of rank $r' < r$ whereas S_1 contains calls to procedures of rank r. According to optimize, statement S_0 is kept in the contextual circuit until no more statements are recursive relative to proc. The statements of rank r' which are present in S_0 are then merged with the equivalent procedures that were derived from S_1 and also added to l_{M}. The number of procedures of rank r' is bounded asymptotically by the maximum between those in S_0 and S_1, therefore we obtain our result.

◀

Theorem 11 (Soundness and Completeness). [Restated, see original statement.]

$[\![\mathrm{pbp}]\!]_{\geq \frac{2}{3}} = \mathrm{fbqp}$.

Proof.

Since $\mathrm{pbp} \subsetneq \mathrm{wf} \cap \mathrm{width}_{\leq 1} \subsetneq \mathrm{pfoq}$, with pfoq the language of [17], we have that $[\![\mathrm{pbp}]\!] \subseteq [\![\mathrm{wf} \cap \mathrm{width}_{\leq 1}]\!] \subseteq [\![\mathrm{pfoq}]\!]$ and, by pfoq soundness [17, Theorem 3], it also holds that $[\![\mathrm{pbp}]\!]_{\geq \frac{2}{3}} \subseteq \mathrm{fbqp}$. Completeness can be proven by showing that pbp can simulate the function algebra in [29], known to be complete for quantum polynomial time. The proof can be done using the same construction as in [17, Theorem 5]. ◀

See 12

Proof.

Any permutation can be written as the composition of two sets of disjoint transpositions, and therefore any permutation can be performed in constant time, using two time steps [24]. To perform a controlled permutation, it suffices to create $O(n)$ ancillas with the correct controlled state, which can be done in $O(\log n)$ depth with $O(n)$ gates. ◀

References

References[1] Samson Abramsky. No-cloning in categorical quantum mechanics. Semantic Techniques in Quantum Computation, pages 1–28, 2009. [2] Leonard M. Adleman, Jonathan DeMarrais, and Ming-Deh A. Huang. Quantum computability. SIAM Journal on Computing, 26(5):1524–1540, 1997. doi:10.1137/S0097539795293639. [3] Thorsten Altenkirch and Jonathan Grattage. A functional quantum programming language. In 20th Annual IEEE Symposium on Logic in Computer Science (LICS' 05), pages 249–258, 2005. doi:10.1109/LICS.2005.1. [4] John Baez. Quantum quandaries: a category-theoretic perspective. The structural foundations of quantum gravity, pages 240–265, 2006. [5] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. Elementary gates for quantum computation. Phys. Rev. A, 52:3457–3467, November 1995. doi:10.1103/PhysRevA.52.3457. [6] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. SIAM Journal on Computing, 26(5):1411–1473, 1997. doi:10.1137/S0097539796300921. [7] Hans J. Briegel, David E. Browne, Wolfgang Dür, Robert Raussendorf, and Maarten Van den Nest. Measurement-based quantum computation. Nature Physics, 5(1):19–26, 2009. doi:10.1038/nphys1157. [8] Andrew M. Childs and Nathan Wiebe. Hamiltonian simulation using linear combinations of unitary operations. Quantum Info. Comput., 12(11–12):901–924, November 2012. doi:10.26421/QIC12.11-12-1. [9] Andrea Colledan and Ugo Dal Lago. Circuit width estimation via effect typing and linear dependency. In European Symposium on Programming, pages 3–30. Springer, 2024. doi:10.1007/978-3-031-57267-8_1. [10] Andrea Colledan and Ugo Dal Lago. Flexible Type-Based Resource Estimation in Quantum Circuit Description Languages. Proc. ACM Program. Lang., 9(POPL), January 2025. doi:10.1145/3704883. [11] Ugo Dal Lago, Andrea Masini, and Margherita Zorzi. Quantum implicit computational complexity. Theoretical Computer Science, 411(2):377–409, 2010. doi:10.1016/j.tcs.2009.07.045. [12] Vincent Danos and Elham Kashefi. Determinism in the one-way model. Physical Review A, 74(5):052310, 2006. doi:10.1103/PhysRevA.74.052310.

[13] Niel de Beaudrap, Xiaoning Bian, and Quanlong Wang. Fast and Effective Techniques for T-Count Reduction via Spider Nest Identities. In 15th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2020), pages 11:1–11:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPICS.TQC.2020.11. [14] Alejandro Díaz-Caro, Mauricio Guillermo, Alexandre Miquel, and Benoît Valiron. Realizability in the unitary sphere. In 2019 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS), pages 1–13. IEEE, 2019. doi:10.1109/LICS.2019.8785834. [15] Peng Fu, Kohei Kishida, and Peter Selinger. Linear dependent type theory for quantum programming languages: Extended abstract. In Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '20, pages 440–453, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3373718.3394765. [16] David Gosset, Robin Kothari, and Kewen Wu. Quantum state preparation with optimal T-count. arXiv preprint arXiv:2411.04790, 2024. doi:10.48550/arXiv.2411.04790. [17] Emmanuel Hainry, Romain Péchoux, and Mário Silva. A programming language characterizing quantum polynomial time. In Foundations of Software Science and Computation Structures, pages 156–175. Springer, 2023. doi:10.1007/978-3-031-30829-1_8. [18] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum Algorithm for Linear Systems of Equations. Physical Review Letters, 103(15), October 2009. doi:10.1103/physrevlett.103.150502. [19] Chris Heunen and Jamie Vicary. Categories for Quantum Theory: an introduction. Oxford University Press, 2019. [20] Aleks Kissinger and John van de Wetering. Reducing the number of non-Clifford gates in quantum circuits. Phys. Rev. A, 102:022406, August 2020. doi:10.1103/PhysRevA.102.022406. [21] Emanuel Knill, Raymond Laflamme, and Gerard J. Milburn. A scheme for efficient quantum computation with linear optics. Nature, 409(6816):46–52, January 2001. doi:10.1038/35051009. [22] Gui-Lu Long and Yang Sun. Efficient scheme for initializing a quantum register with an arbitrary superposed state. Phys. Rev. A, 64:014303, June 2001. doi:10.1103/PhysRevA.64.014303. [23] Dmitri Maslov, Gerhard W. Dueck, D. Michael Miller, and Camille Negrevergne. Quantum Circuit Simplification and Level Compaction. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 27(3):436–444, March 2008. doi:10.1109/tcad.2007.911334. [24] Cristopher Moore and Martin Nilsson. Parallel Quantum Computation and Quantum Codes. SIAM Journal on Computing, 31(3):799–815, 2001. doi:10.1137/S0097539799355053. [25] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. Automated optimization of large quantum circuits with continuous parameters. npj Quantum Information, 4(1):23, May 2018. doi:10.1038/s41534-018-0072-4. [26] Michael A. Nielsen and Isaac L. Chuang. Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press, 2011. [27] Peter Selinger. Towards a quantum programming language. Mathematical Structures in Computer Science, 14(4):527–586, 2004. doi:10.1017/S0960129504004256. [28] Xiaoming Sun, Guojing Tian, Shuai Yang, Pei Yuan, and Shengyu Zhang. Asymptotically optimal circuit depth for quantum state preparation and general unitary synthesis. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 42(10):3301–3314, 2023. doi:10.1109/TCAD.2023.3244885. [29] Tomoyuki Yamakami. A Schematic Definition of Quantum Polynomial Time Computability. J. Symb. Log., 85(4):1546–1587, 2020. doi:10.1017/jsl.2020.45. [30] Tomoyuki Yamakami. Expressing Power of Elementary Quantum Recursion Schemes for Quantum Logarithmic-Time Computability. In Agata Ciabattoni, Elaine Pimentel, and Ruy J. G. B. de Queiroz, editors, Logic, Language, Information, and Computation, pages 88–104, Cham, 2022. Springer International Publishing. doi:10.1007/978-3-031-15298-6_6. [31] Andrew Chi-Chih Yao. Quantum circuit complexity. In Proceedings of 1993 IEEE 34th Annual Foundations of Computer Science, pages 352–361, 1993. doi:10.1109/SFCS.1993.366852. [32] Charles Yuan and Michael Carbin. Tower: Data Structures in Quantum Superposition. Proceedings of the ACM on Programming Languages, 6(OOPSLA2):259–288, October 2022. doi:10.1145/3563297. [33] Charles Yuan, Agnes Villanyi, and Michael Carbin. Quantum control machine: The limits of control flow in quantum programming. Proc. ACM Program. Lang., 8(OOPSLA1), April 2024. doi:10.1145/3649811. [34] Xiao-Ming Zhang, Tongyang Li, and Xiao Yuan. Quantum State Preparation with Optimal Circuit Depth: Implementations and Applications. Phys. Rev. Lett., 129:230504, November 2022. doi:10.1103/PhysRevLett.129.230504. [35] Zhicheng Zhang and Mingsheng Ying. Quantum Register Machine: Efficient Implementation of Quantum Recursive Programs, 2024. doi:10.48550/arXiv.2408.10054.